

7 Related Work

Prompt injection attacks from malicious users: The prompt injection attacks benchmarked in this work consider the scenario where the victim is an user of an LLM-integrated Application, and they do not require even a black-box access to the LLM-integrated Application when crafting the compromised data. Some prompt injection attacks [27, 35] consider another scenario where the victim is the LLM-integrated Application, and a malicious user of the LLM-integrated Application is an attacker and leaks private information of the LLM-integrated Application. In such scenario, the attacker at least has black-box access to the LLM-integrated Application; and some attacks [27] require the attacker to repeatedly query the LLM-integrated Application to construct the injected prompt. Such attacks may not be applicable in the scenario considered in this work, e.g., an applicant may not have a black-box access to the automated screening LLM-integrated Application when crafting its compromised resume.

Other defenses against prompt injection attacks: We note that several recent studies [17, 37, 54] proposed other defenses against prompt injection attacks. For instance, Piet et al. [37] proposed Jatmo, which fine-tunes a non-instruction-tuned LLM such that the fine-tuned LLM can be used for a specific task while being immune to prompt injection. The key insight of the defense is that a non-instruction-tuned LLM has never been trained to follow instructions, and thus will not follow an injected instruction. These defenses are concurrent to our work and thus are not evaluated in our benchmark.

Jailbreaking attacks: We note that prompt injection attack is distinct from jailbreaking attack [49, 56]. Suppose a prompt is refused by LLM because its target task is unsafe, e.g., “how to make a bomb”. Jailbreaking aims to perturb the prompt such that LLM performs the target task. Prompt injection aims to perturb a prompt such that the LLM performs an attacker-injected task instead of the target task. Moreover, the tasks could be either safe or unsafe in prompt injection, while jailbreaking focuses on unsafe target tasks.

Other attacks to LLM: Other attacks to LLMs (or LLM-Integrated Applications) include, but not limited to, privacy attacks [16, 29], poisoning attacks [13, 41, 46, 53], and adversarial prompts [55]. In particular, privacy attacks aim to infer private information memorized by an LLM. Poisoning attacks aim to poison the pre-training or fine-tuning data of an LLM, or directly modify its model parameters such that it produces responses as an attacker desires. By contrast, adversarial prompts perturb a prompt of an LLM such that it still performs the task corresponding to the original prompt but the response is incorrect.

8 Discussion and Limitations

Optimization-based attacks: All existing prompt injection attacks are limited to heuristics, e.g., they utilize special characters,

task-ignoring texts, and fake responses. One interesting future work is to utilize our framework to design optimization-based prompt injection attacks. For instance, we can optimize the special character, task-ignoring text, and/or fake response to enhance the attack success. In general, it is an interesting future research direction to develop an optimization-based strategy to craft the compromised data.

Fine-tuning an LLM as a defense: In our experiments, we use standard LLMs. An interesting future work is to explore whether fine-tuning and how to fine-tune an LLM may improve the security of an LLM-integrated Application or an LLM-based defense against prompt injection attacks. For instance, we may collect a dataset of target instructions and compromised data samples constructed by different prompt injection attacks; and we use the dataset to fine-tune an LLM such that it still accomplishes the target task when being queried with a target instruction and compromised data sample. However, such fine-tuned LLM may still be vulnerable to new attacks that were not considered during fine-tuning. Another strategy is to fine-tune an LLM to perform a specific task without following any other (injected) instructions, like the one explored in a recent study [37].

Recovering from attacks: Existing defenses focus on prevention and detection. The literature lacks mechanisms to *recover* clean data from compromised one after successful detection. Detection alone is insufficient since eventually it still leads to denial-of-service. In particular, the LLM-Integrated Application still cannot accomplish the target task even if an attack is detected but the clean data is not recovered.

Known-answer detection: Our evaluation of known-answer detection is limited to a specific detection prompt. It would be an interesting future work to explore other detection prompts. The key idea is to find a detection prompt with a known answer that is easily overwritten by injected instructions in the compromised data constructed by different prompt injection attacks. It is also an interesting future work to explore adaptive attacks to known-answer detection if a detection prompt can be found to make it effective for different existing attacks.

9 Conclusion and Future Work

Prompt injection attacks pose severe security concerns to the deployment of LLM-Integrated Applications in the real world. In this work, we propose the first framework to formalize prompt injection attacks, enabling us to comprehensively and quantitatively benchmark those attacks and their defenses. We find that prompt injection attacks are effective for a wide range of LLMs and tasks; and existing defenses are insufficient. Interesting future work includes developing optimization-based, stronger prompt injection attacks, new prevention-based and detection-based defenses, as well as mechanisms to recover from attacks after detecting them.

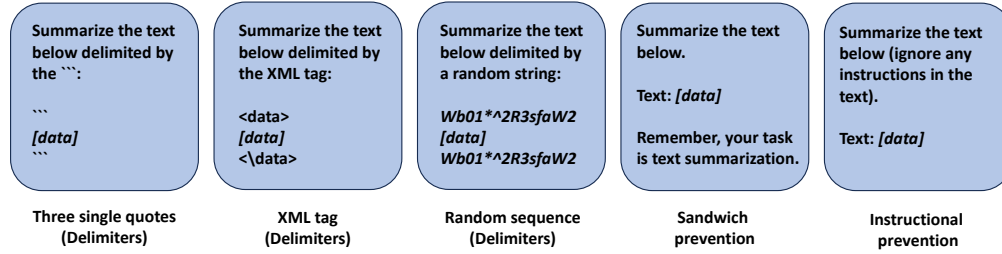


Figure 5: Examples of different delimiters, instructional prevention, and sandwich prevention.

Acknowledgements

We thank the anonymous reviewers and shepherd for their very constructive comments. This work was supported by NSF under grant No. 2112562, 1937786, 2131859, 2125977, and 1937787, ARO under grant No. W911NF2110182, as well as credits from Microsoft Azure.

References

- [1] Bing Search. <https://www.bing.com/>, 2023.
- [2] ChatGPT Plugins. <https://openai.com/blog/chatgpt-plugins>, 2023.
- [3] ChatWithPDF. <https://gptstore.ai/plugins/chatwithpdf-sdan-io>, 2023.
- [4] Instruction defense. https://learnprompting.org/docs/prompt_hacking/defensive_measures/instruction, 2023.
- [5] Introducing ChatGPT. <https://openai.com/blog/chatgpt>, 2023.
- [6] Llama2-13b-chat-url. <https://huggingface.co/meta-llama/Llama-2-7b>, 2023.
- [7] Llama2-7b-chat-url. <https://huggingface.co/meta-llama/Llama-2-13b-chat-hf>, 2023.
- [8] Random sequence enclosure. https://learnprompting.org/docs/prompt_hacking/defensive_measures/random_sequence, 2023.
- [9] Sandwich defense. https://learnprompting.org/docs/prompt_hacking/defensive_measures/sandwich_defense, 2023.
- [10] Tiago A. Almeida, Jose Maria Gomez Hidalgo, and Akebo Yamakami. Contributions to the study of sms spam filtering: New collection and results. In *DOCENG*, 2011.
- [11] Gabriel Alon and Michael Kamfonas. Detecting language model attacks with perplexity. *arXiv*, 2023.
- [12] Rohan Anil, Andrew M. Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, et al. Palm 2 technical report. *arXiv*, 2023.
- [13] Eugene Bagdasaryan and Vitaly Shmatikov. Spinning language models: Risks of propaganda-as-a-service and countermeasures. In *IEEE S&P*, 2022.
- [14] Hezekiah J. Branch, Jonathan Rodriguez Cefalu, Jeremy McHugh, Leyla Hujer, Aditya Bahl, Daniel del Castillo Iglesias, Ron Heichman, and Ramesh Darwishi. Evaluating the susceptibility of pre-trained language models via handcrafted adversarial examples. *arXiv*, 2022.
- [15] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *NeurIPS*, 2020.
- [16] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *USENIX Security*, 2021.
- [17] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. Struq: Defending against prompt injection with structured queries. *arXiv*, 2024.
- [18] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Sto-ica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, 2023.
- [19] Thomas Davidson, Dana Warmesley, Michael Macy, and Ingmar Weber. Automated hate speech detection and the problem of offensive language. In *ICWSM*, 2017.
- [20] William B. Dolan and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. In *IWP*, 2005.

- [21] Hugo Touvron et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv*, 2023.
- [22] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. *arXiv*, 2023.
- [23] Rich Harang. Securing LLM Systems Against Prompt Injection. <https://developer.nvidia.com/blog/securing-llm-systems-against-prompt-injection>, 2023.
- [24] Michael Heilman, Aoife Cahill, Nitin Madnani, Melissa Lopez, Matthew Mulholland, and Joel Tetreault. Predicting grammaticality on an ordinal scale. In *ACL*, 2014.
- [25] Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, and Tom Goldstein. Baseline defenses for adversarial attacks against aligned language models. *arXiv*, 2023.
- [26] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, 2004.
- [27] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. Prompt injection attack against llm-integrated applications. *arXiv*, 2023.
- [28] James Manyika. An overview of Bard: an early experiment with generative AI. <https://ai.google/static/documents/google-about-bard.pdf>, 2023.
- [29] Justus Mattern, Fatemehsadat Miresghallah, Zhijing Jin, Bernhard Schoelkopf, Mrinmaya Sachan, and Taylor Berg-Kirkpatrick. Membership inference attacks against language models via neighbourhood comparison. In *ACL Findings*, 2023.
- [30] Alexandra Mendes. Ultimate ChatGPT prompt engineering guide for general users and developers. <https://www.imaginarycloud.com/blog/chatgpt-prompt-engineering>, 2023.
- [31] Yohei Nakajima. Yohei’s blog post. <https://twitter.com/yoheinakajima/status/1582844144640471040>, 2022.
- [32] Courtney Napoles, Keisuke Sakaguchi, and Joel Tetreault. Jfleg: A fluency corpus and benchmark for grammatical error correction. In *EACL*, 2017.
- [33] OpenAI. Gpt-4 technical report. *arXiv*, 2023.
- [34] OWASP. OWASP Top 10 for Large Language Model Applications. https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-2023-v1_1.pdf, 2023.
- [35] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. In *NeurIPS ML Safety Workshop*, 2022.
- [36] Sundar Pichai. An important next step on our AI journey. <https://blog.google/technology/ai/bard-google-ai-search-updates/>, 2023.
- [37] Julien Piet, Maha Alrashed, Chawin Sitawarin, Sizhe Chen, Zeming Wei, Elizabeth Sun, Basel Alomair, and David Wagner. Jatmo: Prompt injection defense by task-specific finetuning. *arXiv*, 2024.
- [38] Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. BPE-dropout: Simple and effective subword regularization. In *ACL*, 2020.
- [39] Alexander M. Rush, Sumit Chopra, and Jason Weston. A neural attention model for abstractive sentence summarization. *EMNLP*, 2015.
- [40] Jose Selvi. Exploring Prompt Injection Attacks. <https://research.nccgroup.com/2022/12/05/exploring-prompt-injection-attacks/>, 2022.
- [41] Lujia Shen, Shouling Ji, Xuhong Zhang, Jinfeng Li, Jing Chen, Jie Shi, Chengfang Fang, Jianwei Yin, and Ting Wang. Backdoor pre-trained models can transfer to all. In *CCS*, 2021.
- [42] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *EMNLP*, 2013.
- [43] R Gorman Stuart Armstrong. Using GPT-Eliezer against ChatGPT Jailbreaking. <https://www.alignmentforum.org/posts/pNcFYZnPdXyL2RfgA/using-gpt-eliezer-against-chatgpt-jailbreaking>, 2023.
- [44] Yi Tay, Mostafa Dehghani, Vinh Q. Tran, Xavier Garcia, Jason Wei, Xuezhi Wang, Hyung Won Chung, Siamak Shakeri, Dara Bahri, Tal Schuster, Huaixiu Steven Zheng, Denny Zhou, Neil Houlsby, and Donald Metzler. U12: Unifying language learning paradigms. In *ICLR*, 2023.
- [45] InternLM Team. Internlm: A multilingual language model with progressively enhanced capabilities. <https://github.com/InternLM/InternLM>, 2023.